

Project Initialization and Planning Phase

Date	04 July 2026
Team ID	
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	5 Marks

Project Proposal (Proposed Solution) Report

This proposal outlines a machine learning based flood prediction system that classifies flood risk from historical weather and rainfall readings, deployed through a Flask web application for instant, self-serve predictions.

Project Overview

Field	Details
Objective	Give users an instant, data-driven flood risk read from local weather and rainfall readings, without needing to interpret raw meteorological data themselves.
Scope	Covers model training/comparison (offline, in a notebook), and a deployed Flask web app that collects readings, scales them, runs inference, and displays a risk result with a visual gauge.

Problem Statement

Field	Details
Description	Flood risk assessment today depends on manual monitoring of rainfall/river data and infrequent official bulletins, leaving residents without a fast, localized way to gauge risk.
Impact	Delays in recognizing rising flood risk can mean less time to prepare, evacuate, or protect property; a fast automated read shortens that reaction gap.

Proposed Solution

Field	Details
Approach	Train and compare multiple classification models (Decision Tree, Random Forest, KNN, Gradient Boosting) on historical rainfall/weather data, select the best performer, and serve it through a lightweight Flask app.
Key Features	- 10-field weather/rainfall input form. - Feature scaling identical to training, via a saved scaler. - Instant flood / no-flood classification. - Animated risk gauge result page with light/dark theme.

Resource Requirements

Resource Type	Description	Specification / Allocation
Hardware - Computing Resources	CPU specifications for inference (no GPU needed for these classical ML models)	Standard CPU (local dev machine / Render's shared instance)
Hardware - Memory	RAM specifications	512 MB - 1 GB (sufficient for Flask + a small serialized model)
Hardware - Storage	Disk space for model artifacts, code, and static assets	< 50 MB (floods.save, transform.save, templates, static files)

Resource Type	Description	Specification / Allocation
Software - Framework	Python web framework	Flask 3.0
Software - Libraries	Core ML/data libraries	scikit-learn, pandas, numpy, joblib, gunicorn
Software - Dev Environment	Where the model was trained and the app	Jupyter Notebook (training), VS Code (Flask)

	developed	app)
Data	Source, size, format	Historical rainfall & weather dataset (flood dataset.xlsx), 10 numeric features, binary flood label